

Calculadora Mejorada

INSTRUCCIONES

Crear una calculadora en Python que funcione desde la consola. La calculadora debe tener estas 4 funciones (con estos nombres exactos):

- `addmultiplenumbers(lista)`
 - a. Suma todos los números de una lista y regresa el resultado.
 - `multiplymultiplenumbers(lista)`
 - a. Multiplica todos los números de una lista y regresa el resultado.
 - `isiteven(numero)`
 - a. Regresa True si el número es par (y entero), o False si no.)
 - `isitaninteger(numero)`
 - a. Regresa True si el número es entero (por ejemplo 7 o 7.0), o False si no.
-

REGLAS DEL TRABAJO

- Usar una función principal para arrancar el programa.
- Dividir el código en funciones (no todo junto).
- Aceptar números positivos y negativos.
- Pensar en casos especiales (lista vacía, números decimales).
- Mensajes claros para el usuario.

Cómo trabajar con el repositorio (GitHub):

1. Hacer Fork del repositorio original.
2. Clonar su Fork a su computadora.
3. Crear una rama de trabajo.
4. Subir cambios y hacer Pull Request.

Herramientas necesarias:

- Python 3 instalado.
- Librerías: pytest, pytest-cov, black, flake8, isort, mypy (estas se instalan con pip install).
- Extensiones VS Code: Python, Pylance, Black Formatter, isort, GitHub Pull Requests and Issues.

main.py

Instalación de librerías

En una consola, ubicada en la misma carpeta del proyecto, usar el siguiente comando para instalar las librerías:

```
pip install pytest pytest-cov black flake8 isort mypy
```

Definen las funciones

- addmultiplenumbers(lista)
- multiplymultiplenumbers(lista)
- isiteven(numero)
- isitaninteger(numero)

```
# Función que suma todos los elementos de una lista.
# Si la lista está vacía, devuelve 0.
def addmultiplenumbers(lista):
    if not lista:
        return 0
    return sum(lista)

# Función que multiplica todos los elementos de una lista.
# Si la lista está vacía, devuelve 1 (identidad multiplicativa).
def multiplymultiplenumbers(lista):
    if not lista:
        return 1
    result = 1
    for num in lista:
        result *= num
    return result

# Función que verifica si un número es par.
# Utiliza la función isitaninteger para validar primero si el número es entero, y luego verificar si es par
def isiteven(numero):
    return isitaninteger(numero) and numero % 2 == 0

# Función que verifica si un número es entero.
# Acepta tanto enteros como flotantes sin decimales (ej. 3.0).
def isitaninteger(numero):
    return isinstance(numero, int) or (
        isinstance(numero, float) and numero.is_integer()
    )
```

Despliegue del menú

- Se define la función `main` que se usará para las pruebas
- La función `main` contiene el menú de las operaciones
- Se solicita que el usuario elija una opción

```
# Función principal que interactúa con el usuario.  
# Permite elegir entre suma, multiplicación, verificación de paridad o entero.  
def main():  
    print("Bienvenido a la Calculadora Mejorada ")  
    print("Opciones: \n" \  
        "1- Suma lista de números\n" \  
        "2- Multiplicación lista de números\n" \  
        "3- Verificar si el número es par \n" \  
        "4- Verificar si el número es entero\n")  
  
    # Solicita al usuario la operación deseada  
    opcion = input("¿Qué operación deseas realizar? ")
```

Ejecución de funciones

- Se usa la estructura de control `if` para determinar el despliegue de la función
- Cada función tiene asignado un número que usará el usuario para elegir
- Si se elige 1, despliega suma, si se elige 2 la multiplicación
 - Para cualquiera de estos dos casos se solicita una lista de números
 - Se separan los números por comas para elaborar la lista
- La opción 3 despliega la identificación de número par y solicita insertar un número
- La opción 4 despliega la identificación de número entero y solicita insertar un número
- Para cualquier caso se aceptan números con o sin punto decimal
- Si se inserta un valor/opción que no esté dentro del menú se manda mensaje de error

```
# Si la opción es suma o multiplicación, pide una lista de números
if opcion in ["1", "2"]:
    entrada = input("Introduce los números separados por coma: ")
    lista = [float(x) for x in entrada.split(",") if x.strip()]
    if opcion == "1":
        print("Resultado:", addmultiplenumbers(lista))
    else:
        print("Resultado:", multiplymultiplenumbers(lista))

# Si la opción es paridad, pide un número y verifica si es par
elif opcion == "3":
    numero = float(input("Introduce un número: "))
    print("¿Es par?", isiteven(numero))

# Si la opción es entero, verifica si el número ingresado es entero
elif opcion == "4":
    numero = float(input("Introduce un número: "))
    print("¿Es entero?", isitaninteger(numero))

# Si la opción no es válida, muestra un mensaje de error
else:
    print("Opción no válida.")
```

Despliegue de main

- En caso de ejecutar el archivo `main.py` esto hará que despliegue la función main

```
if __name__ == "__main__":
    main()
```

test.py

Desde el archivo `main.py` se importaran las funciones:

- `addmultiplenumbers(lista)`
- `multiplymultiplenumbers(lista)`
- `isiteven(numero)`
- `isitaninteger(numero)`

```
from main import (addmultiplenumbers, isitaninteger, isiteven,  
                  multiplymultiplenumbers)
```

Se definen las pruebas que se haran al archivo `main.py`

```
# Test 5  
def test_is_six_even():  
    response = isiteven(6)  
    assert response == True  
  
# Test 6  
def test_is_minus_three_point_eight_even():  
    response = isiteven(-3.8)  
    assert response == False  
  
# Test 7  
def test_is_three_an_integer():  
    response = isitaninteger(3)  
    assert response == True  
  
# Test 8  
def test_is_seven_point_three_an_integer():  
    response = isitaninteger(7.3)  
    assert response == False
```

```
# Test 1  
def test_add_multiple_positive_integers():  
    response = addmultiplenumbers([5, 7, 9])  
    assert response == 21  
  
# Test 2  
def test_add_multiple_complicated_numbers():  
    response = addmultiplenumbers([5, -7, 9.3])  
    response = round(response, 1)  
    assert response == 7.3  
  
# Test 3  
def test_multiply_multiple_positive_integers():  
    response = multiplymultiplenumbers([4, 5, 6, 7])  
    assert response == 840  
  
# Test 4  
def test_multiply_multiple_complicated_numbers():  
    response = multiplymultiplenumbers([4, -5, 6.7])  
    response = round(response, 0)  
    assert response == -134
```

Uso de herramientas

black

Formato automático de código. Aplica un estilo consistente, eliminando debates sobre el formato.

Para usarse se escribe *black*.

Automáticamente aplica el formato a los archivos en la carpeta

```
PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejora
da\TIFC1-PF-3-3-1-BetterCalculator-Individual> black .
reformatted C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculado
ra mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual\test_.py
reformatted C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculado
ra mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual\main.py
```

All done! ✨ 🍰 ✨

2 files reformatted.

```
PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejora
da\TIFC1-PF-3-3-1-BetterCalculator-Individual> 
```

<i>flake8</i> Analiza el código en busca de errores de estilo, sintaxis y convenciones PEP8. Ideal para mantener la calidad. Para usarse escribe <i>flake8</i> Automáticamente revisa los archivos y sugiere cambios En este caso sugiere cambios que hacen que el archivo test no funcione como debería por lo que se ignoraron las sugerencias además de sugerir acortar líneas muy largas o revertir cambios que el comando <i>black</i> hizo	<pre>PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> flake8 .\main.py:21:80: E501 line too long (108 > 79 characters) .\test_.py:1:80: E501 line too long (85 > 79 characters) .\test_.py:33:21: E712 comparison to True should be 'if cond is True:' or 'if cond:' .\test_.py:39:21: E712 comparison to False should be 'if cond is False:' or 'if not cond:' .\test_.py:45:21: E712 comparison to True should be 'if cond is True:' or 'if cond:' .\test_.py:51:21: E712 comparison to False should be 'if cond is False:' or 'if not cond:' PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual></pre>
<i>isort</i> Ordena automáticamente los imports según reglas definidas. Mejora la legibilidad y evita duplicados. Para usar se escribe <i>isort <nombre del archivo></i> automáticamente analiza y modifica lo necesario	<pre>da\TIFC1-PF-3-3-1-BetterCalculator-Individual> isort test_.py Fixing C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual\test_.py PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual></pre>

<i>mypy</i> Verifica tipos estáticos en Python. Detecta errores de tipo antes de ejecutar el código, útil en proyectos grandes. Para usarse solo se escribe <i>mypy <nombre de archivo></i> en la terminal de comandos automáticamente analiza y alerta de los errores	<pre>PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> mypy main.py Success: no issues found in 1 source file PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> mypy test_.py Success: no issues found in 1 source file PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> </pre>
<i>pytest</i> Ejecuta pruebas unitarias y funcionales. Permite validar que el código funciona como se espera. Para usarse solo se escribe <i>pytest</i> en la terminal de comandos	<pre>PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> pytest ===== test session starts ===== platform win32 -- Python 3.13.5, pytest-8.4.1, pluggy-1.6.0 rootdir: C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual plugins: cov-6.2.1 collected 8 items test_.py [100%] ===== 8 passed in 0.08s ===== PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual> </pre>

`pytest-cov`

Extiende pytest para medir la cobertura de código: qué partes del código fueron ejecutadas durante las pruebas.

Para usarse solo se escribe `pytest --cov` en la terminal de comandos

Automáticamente analiza los archivos y suelta información de ellos

```
PS C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora mejora
da\TIFC1-PF-3-3-1-BetterCalculator-Individual> pytest --cov
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\jon11\OneDrive\Escritorio\CH3 code\calculadora
mejorada\TIFC1-PF-3-3-1-BetterCalculator-Individual
plugins: cov-6.2.1
collected 8 items

test_.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.13.5-final-0 -----

Name          Stmts   Miss  Cover
-----
main.py         34     19    44%
test_.py        27      0   100%
-----
TOTAL           61     19    69%
===== 8 passed in 0.12s =====
```

Uso de IA

Se hizo uso de IA para la investigación sobre el uso de librerías como pytest y black, así como la definición de formatos de operación del proyecto. Esta facilitó la exploración de buenas prácticas, la automatización de pruebas y la estructuración de documentación técnica clara y coherente.